RHYTHM

A Music Based Programming Language: README

Course	Compiler Construction
Members	Mehrmah Khan (23L 0623), Esha Irfan (23L 0836)

1. What this is

RHYTHM is a programming language for writing music. A program has variables, loops, functions and lists, and it declares tracks of notes. Running a program prints a timeline and writes a WAV sound file.

The project has two phases built from the same Flex lexer and Bison grammar. **Phase 1** is an interpreter (`bin/rhythmi`): the C code inside `src/interp.y` runs the program, with no intermediate code. **Phase 2** is a compiler (`rhythmc`): the C code inside `src/compiler.y` emits LLVM IR, and clang turns it into a native executable. Both phases give the same output, and the test suite checks that byte for byte.

2. Installing the dependencies

Ubuntu or WSL (Ubuntu) on Windows. Run:

```
sudo apt update
sudo apt install build-essential flex bison clang llvm
```

LLVM 15 or newer is required because the generated IR uses opaque pointers. It was tested with LLVM and clang 18.

3. Building and testing

Unpack the project, open a terminal inside the folder and run:

```
make         # builds bin/rhythmi, bin/rhythm_ir and bin/librhythm_rt.a
make test    # runs all examples and tests against both phases
make clean   # removes everything that was built
```

A good result ends with `passed 68, failed 0`.

4. Running a program

Phase 1, the interpreter

```
./bin/rhythmi examples/02_arpeggio.rhy -o song.wav
./bin/rhythmi program.rhy -n        # print the timeline, write no audio
```

Phase 2, the compiler

```
./rhythmc examples/02_arpeggio.rhy -o song    # writes ./song and song.ll
./song -o song.wav                           # run the native program
```

`rhythmc` runs `bin/rhythm_ir` to get LLVM IR, then clang compiles it and links `bin/librhythm_rt.a`. The IR is kept as `song.ll` so it can be read. To listen, open the WAV in an audio player. In WSL, copy it to Windows first, for example `cp song.wav /mnt/c/Users/YOURNAME/Desktop/`.

5. The language

A program is a list of statements. Statements end in a semicolon, blocks use braces. Keywords are lower case. Comments are `//`, `#` and `/* ... */`.

Values

There are two kinds of value: numbers and lists of numbers. A note name such as `C4`, `F#3` or `Bb5` is a number (the MIDI number, so C4 is 60). Because of that, `C4 + 7` is G4, and a scale is just a list of offsets. A zero (or not a number) is false, anything else is true.

Statements

Area	Syntax
Variables	`let x = 5;` `x = 6;` `x += 1;` (also `-=`, `*=`, `/=`) `xs[i] = v;`
Conditions	`if c { } else if c { } else { }`
Loops	`while c { }` `for i in 0..8 { }` `for i in 0..8 by 2 { }` `for x in list { }` `repeat 4 times { }`
Loop control	`stop;` leaves the loop, `skip;` goes to the next round
Functions	`fn name(a, list b) { ... return a; }` A parameter written `list b` must receive a list.
Tracks	`track name { ... }` holds music statements and may use loops, ifs and calls. `return;` ends it early.
Music	`tempo 120;` `note C4 for 2 beats;` `rest for 1 beat;` `chord [C4, E4, G4] for 2 beats;` `instrument saw;` `volume 70;`
Playing	`play a, b * 2;` plays tracks together and repeats b twice. `play a to "x.wav";` picks the file. `play;` plays every track.
Output	`say "text", value, other;`

Expressions

Operators from lowest to highest precedence: `or`, `and`, `not`, comparisons (`< <= > >= == !=`), `+ -`, `* / %`, unary minus. `and` and `or` stop early. Lists are written `[1, 2, 3]` and read with `xs[i]`. Built in functions:

Function	Meaning
`len(xs)`	number of items
`push(xs, v)` `pop(xs)`	add to the end, remove and return the last item
`zeros(n)` `copy(xs)`	list of n zeros, an independent copy of a list
`floor` `abs` `sqrt`	one argument math
`min` `max` `pow`	two argument math
`rand(n)` `seed(n)`	whole number from 0 to n minus 1, repeatable when seeded

Instruments

`sine`, `square`, `saw` and `triangle` select the wave used for a track.

Errors

Before anything runs, the program is checked: undeclared names, wrong argument counts, a list used where a number is needed, statements in the wrong place (`note` outside a track, `stop` outside a loop), constants out of range (tempo 20 to 400, volume 0 to 100, note 0 to 127, duration above 0 and up to 64). Messages look like `file:line: message` and the exit status is 1. Problems that depend on values, such as a list index out of range or division by zero, stop the program with `file:line: runtime error: ...`, also with status 1. Both phases print identical messages.

6. Example programs

Each file is in `examples/`. Run any of them with either phase.

File	What it shows
`01_hello.rhy`	the smallest program
`02_arpeggio.rhy`	the proposal example: a function, a list, loops and two tracks
`03_twinkle.rhy`	a familiar tune using lists and loops
`04_fibonacci.rhy`	recursion turned into a melody
`05_sorting.rhy`	bubble sort, played before and after sorting
`06_euclid_rhythm.rhy`	Euclid's algorithm used as a rhythm
`07_primes.rhy`	sieve of Eratosthenes as music
`08_random_walk.rhy`	a seeded random melody
`09_language_tour.rhy`	every feature, printed with `say` (no audio)

Example program and its output:

```
tempo 120;
fn arpeggio(root, list steps) {
  for s in steps { note root + s for 0.5 beats; }
}
track melody {
  arpeggio(C4, [0, 4, 7]);
}
play melody;
```

```
[play 1] 1 track, 120 BPM, 1.5 beats (0.75 seconds)
  melody [sine, volume 80] 1.5 beats: C4/0.5 E4/0.5 G4/0.5
  wrote rhythm.wav
```

7. Tests

`make test` runs `tests/run_tests.sh`, which has three parts.

Examples and programs (10 programs): each is run by the interpreter and also compiled and run. The printed output, error output, exit status and every WAV file must be identical.

tests/errors (39 programs): each has a first line `// EXPECT: message`. Both phases must reject the program with exit status 1 and that message, and the compiler must write no IR.

tests/runtime (19 programs): they compile fine but fail while running. Both phases must fail with the same message.

8. Files

File	Purpose
`src/rhythm.l`	Flex lexer: keywords, numbers, note names, operators, comments
`src/interp.y`	Bison grammar with the Phase 1 interpreter in its epilogue
`src/compiler.y`	the same grammar with the Phase 2 LLVM IR generator in its epilogue
`src/ast.h`, `src/ast.c`	syntax tree and the semantic analysis (scopes, types, context rules, constants)

`src/checks.h`	value range rules shared by compile time and run time checks
`src/rtlib.c`, `src/rtlib.h`	runtime library used by the interpreter and by compiled programs
`src/engine.c`, `src/engine.h`	tone synthesis, chords, timeline and WAV writing
`src/common.c`, `src/common.h`	error messages, note name conversion, string buffer
`rhythmc`	shell script: source to IR to executable
`Makefile`	targets all, test and clean
`examples/`	nine example programs
`tests/`	`run_tests.sh`, `programs/`, `errors/`, `runtime/`

9. How it works inside

The lexer turns text into tokens. The Bison grammar builds a syntax tree from the actions on each rule. A semantic pass then resolves every name to a numbered slot, checks that each expression is a number or a list where required, and checks where statements may appear. Nothing runs if this pass reports an error.

In Phase 1 a tree walking evaluator, written in C at the end of `interp.y`, executes the program directly. There is no intermediate code or bytecode. In Phase 2 the C code at the end of `compiler.y` walks the same tree and writes LLVM IR text. Every `fn` becomes `define double @fn.NAME`, every `track` becomes `define void @track.NAME`, and top level statements become `@main`. Numbers are doubles, lists are pointers to runtime lists, `if`, `while`, `for` and `repeat` become basic blocks with branches, and `and` and `or` use a phi node. Anything that depends on run time values is a call into `librhythm_rt.a`, the same library the interpreter uses.